Problem Statement

Universities often struggle with managing course enrollments, recording assessment marks, and providing students with clear insights into their academic performance. Existing systems either lack transparency in grading or do not provide real-time analytics about student standing within a class. The goal of this project is to develop a **microservice-based Academic Performance Management System** that enables administrators to manage courses and enrollments, professors to define evaluation schemes and upload marks, and students to track their academic progress and relative performance. The system will be designed with a **modern DevOps pipeline** to ensure automated building, testing, deployment, scalability, and monitoring.

Microservice	One-Line Responsibility
Auth Service	Handles user authentication, login/logout, and generates JWT tokens for secure access.
User Service	Manages user profiles and roles such as admin, professor, and student.
Course Service	Creates courses and manages student enrollments and professor assignments.
Marks Service	Handles evaluation components, mark uploads, and calculates final scores.
Analytics Service	Computes rankings, class statistics, and performance analytics.
Notification Service	Sends alerts to users when events occur (marks uploaded, course updates, etc.).
API Gateway	Acts as a single entry point that routes frontend requests to the appropriate microservice.
Frontend	Provides the user interface where admins, professors, and students interact with the system.

User Roles and Functionalities

1️⃣ Admin

The admin manages the overall academic structure and user management.

Functionalities

- Create and manage courses and subjects
- Add and manage users (students and professors)
- Enroll students in specific courses
- Assign professors to courses
- Manage academic semesters or course offerings
- Monitor system usage and activity logs

2️⃣ Professor

Professors manage course evaluations and grading for enrolled students.

Functionalities

- View list of students enrolled in their courses
- Define evaluation components (e.g., Midterm, Assignment, Quiz, Final)
- Set weightage for each evaluation component
- Upload marks for individual students or via CSV upload
- View class performance statistics and grade distribution
- Identify top performers and students needing improvement

3️⃣ Student

Students use the system to track their academic progress and performance.

Functionalities

- View enrolled courses
- View marks for each evaluation component
- See calculated final score based on weightage
- View class average and relative class ranking
- Track performance trends across assessments
- Receive notifications when marks or course updates are posted

Language Choice for Each Microservice

You should assign languages based on **what the service does best**.

Microservice	Language	Why
API Gateway	**Golang**	Very fast, lightweight, excellent for handling many HTTP requests
Auth Service	**Spring Boot**	Strong security ecosystem (Spring Security, JWT support)
User Service	**Spring Boot**	CRUD-heavy service where Spring Boot is very efficient
Course Service	**Spring Boot**	Structured business logic fits well with Spring
Marks Service	**Spring Boot**	Handles business logic for grading and weightage
Analytics Service	**Golang**	Faster computation for ranking and statistics
Notification Service	**Golang**	Lightweight service for async tasks
Frontend	**React + Tailwind**	Fast UI development

Recommended Database: PostgreSQL vs MySQL

Use: PostgreSQL

Why PostgreSQL is Better Here

1️⃣ **Better for complex queries**

Your analytics service will need ranking, aggregates, and statistics. PostgreSQL handles these better.

2️⃣ **More advanced features**

Supports window functions, JSON fields, and better indexing.

3️⃣ **Better for microservices and scalability**

PostgreSQL is widely used with Kubernetes and modern architectures.

4️⃣ **Strong consistency and reliability**

✅ **Final choice: PostgreSQL**

Microservice	Functionality	Language	Purpose / Type of Service
API Gateway	Receives all client requests and routes them to the correct backend service.	Golang	Acts as an **entry-point service** that manages routing, authentication checks, and request forwarding between the frontend and microservices.
Auth Service	Handles login, registration, password encryption, and JWT token generation.	Spring Boot	A **security service** responsible for authentication and access control across the system.
User Service	Manages user data including students, professors, and admin profiles.	Spring Boot	A **user management service** responsible for maintaining identity and profile information of all users.
Course Service	Creates courses, assigns professors, and enrolls students.	Spring Boot	A **domain service** that manages academic structure such as courses and student enrollment.
Marks Service	Stores evaluation components, uploads marks, and calculates weighted scores.	Spring Boot	A **grading service** responsible for handling academic assessments and computing final marks.
Analytics Service	Calculates class rankings, averages, and performance insights.	Golang	A **data-processing service** that generates statistics and performance analytics for students and professors.
Notification Service	Sends alerts when marks are uploaded or course updates occur.	Golang	An **event-based service** that informs users about important system events and updates.

Database Design:

1️⃣ Users Table

Stores all users (Admin, Professor, Student).

```sql
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    role VARCHAR(20) CHECK (role IN ('ADMIN','PROFESSOR','STUDENT')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Purpose:

- authentication
- role-based access

2️⃣ Courses Table

Stores course information.

```sql
CREATE TABLE courses (
    id SERIAL PRIMARY KEY,
    course_code VARCHAR(20) UNIQUE NOT NULL,
    course_name VARCHAR(150) NOT NULL,
    professor_id INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (professor_id) REFERENCES users(id)
);
```

Purpose:

- course creation
- professor assignment

3️⃣ Enrollments Table

Maps **students to courses.**

```sql
CREATE TABLE enrollments (
    id SERIAL PRIMARY KEY,
    student_id INT NOT NULL,
    course_id INT NOT NULL,
    enrolled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES users(id),
    FOREIGN KEY (course_id) REFERENCES courses(id),
    UNIQUE(student_id, course_id)
);
```

Purpose:

- student registration in courses

4️⃣ Evaluation Components Table

Defines grading components.

Example:

```
Midterm 30%
Assignment 20%
Final 50%
```

```sql
CREATE TABLE evaluation_components (
    id SERIAL PRIMARY KEY,
    course_id INT NOT NULL,
    component_name VARCHAR(100) NOT NULL,
    weightage INT NOT NULL,
    FOREIGN KEY (course_id) REFERENCES courses(id)
);
```

Purpose:

- professor defines grading structure

5️⃣ Marks Table

Stores student marks per component.

```sql
CREATE TABLE marks (
    id SERIAL PRIMARY KEY,
    student_id INT NOT NULL,
    course_id INT NOT NULL,
    component_id INT NOT NULL,
    marks_obtained FLOAT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES users(id),
    FOREIGN KEY (course_id) REFERENCES courses(id),
    FOREIGN KEY (component_id) REFERENCES evaluation_components(id),
    UNIQUE(student_id, component_id)
);
```

Purpose:

- professor uploads marks

6️⃣ Final Scores Table (Optional but useful)

Stores computed final score.

```sql
CREATE TABLE final_scores (
    id SERIAL PRIMARY KEY,
    student_id INT NOT NULL,
    course_id INT NOT NULL,
    final_score FLOAT,
    calculated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES users(id),
    FOREIGN KEY (course_id) REFERENCES courses(id),
    UNIQUE(student_id, course_id)
);
```

Purpose:

- ranking and analytics

7️⃣ Notifications Table

Stores system notifications.

```sql
CREATE TABLE notifications (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    message TEXT NOT NULL,
    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Purpose:

- alert students when marks uploaded

🧱 Which Microservice Uses Which Tables

Microservice	Tables
Auth Service	users
User Service	users
Course Service	courses, enrollments
Marks Service	evaluation_components, marks
Analytics Service	final_scores
Notification Service	notifications

Final Development Order

```
1 Auth Service
2 User Service
3 Course Service
4 Marks Service
5 Analytics Service
6 Notification Service
7 API Gateway
8 Frontend
```